Stopwatch / Countdown / Pomodoro Timer

- **Problem:** Build a timer app with three modes: a stopwatch with laps, a countdown from a duration the user sets, and a Pomodoro timer that cycles focus and break sessions. Pause / resume must never lose or gain time.
- **Tech stack:** React (Vite, JavaScript, Tailwind CSS v4)
- **Time limit:** 60 minutes
- **Often asked at:** Amazon, Paytm, Zomato
- **Why it matters:** Timer drift, pause / resume

PART A: Thought process

General steps for any machine coding task

1. **Understand the requirements.** Restate the problem, split functional vs non-functional, ask clarifying questions, write down assumptions.
2. **List the interactions.** Every user action and its expected result. This is your checklist for building and testing.
3. **Design the architecture.** Name the components and their one-line jobs. Start small.
4. **Design the state.** What is the single source of truth? What can be derived?
5. **Design data structures and contracts.** Shape of the data, hook inputs and outputs, function signatures.
6. **Trace one data flow end to end.** Proves the design works before you type.
7. **Find the hard parts.** Solve them on paper first.
8. **Build in layers.** Simplest working version first, then one working layer at a time.
9. **Cover states and edge cases.** Idle, running, paused, done, invalid input, background tab.
10. **Wrap up.** Trade-offs and what you would add with more time.

Time budget for 60 minutes

Phase	Time
Requirements + clarifying questions	5 min
Interactions, architecture, state design	7 min
Hard parts on paper (drift, pause/resume, end detection)	5 min
Build: clock reducer + `useClock` + Stopwatch	13 min
Build: laps	5 min
Build: `useCountdown` + Countdown screen	12 min
Build: Pomodoro cycle	8 min
Edge cases, accessibility, wrap-up	5 min

Tip: think out loud

Say what you are deciding and why: "I won't count ticks. I'll store when the timer started and compute elapsed from the clock, so a late `setInterval` can never make the time wrong." The interviewer grades your reasoning as much as the code.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- **Three modes** in tabs: Stopwatch, Countdown, Pomodoro. Switching tabs does **not** stop a running timer.
- **Stopwatch:** Start, Pause, Resume, Reset. Shows `mm:ss.cc` (hours appear after 60 minutes). **Lap** records a lap; the list shows each lap's time, newest first, with the fastest lap in green and the slowest in red.
- **Countdown:** the user sets hours / minutes / seconds. Start, Pause, Resume, Reset. At zero: it stops at `00:00`, plays a beep, shows "Time's up!" and offers Restart.
- **Pomodoro:** 25 min focus, 5 min short break, 15 min long break after every 4 finished focus sessions. Start, Pause, Resume, Reset, Skip. When a phase ends: beep, load the next phase. Shows the phase name, a progress dot per session, the completed count, and the remaining time in the browser tab title.

Non-functional requirements

Area	What it means here
Accuracy	No drift. The time is right after many pause / resume cycles, a busy main thread, or a hidden browser tab.
Performance	Re-render only as often as the display needs (about 30 fps for centiseconds, 5 per second for whole seconds). No timers run while idle or paused.
Error handling	Bad input (`""` , `-3` , `2.7` , `75` minutes) is clamped; a zero duration cannot start; blocked audio never breaks the app.
Accessibility	Native buttons, ARIA tabs, `role="timer"` for the display (it is not read out every tick), `aria-live` for "Time's up" and phase changes, `tabular-nums` so digits do not jitter.
Reusability	The timing engine lives in hooks (`useClock` , `useCountdown`) and pure functions, with no UI inside.
Extensibility	Pomodoro durations come from a `config` prop; a new mode (interval training, chess clock) reuses the same hooks.

Clarifying questions to ask

1. One app with three modes, or three separate widgets? Should a running timer keep going when I switch modes?
2. What precision does the stopwatch need: seconds, tenths, or hundredths?
3. What should happen when the countdown hits zero: sound, browser notification, auto-repeat?

4. Pomodoro: fixed 25 / 5 / 15 or user-configurable? Should the next phase start automatically?
5. Must the timers survive a page reload?
6. Must the time stay correct while the tab is in the background or the laptop is asleep?

Assumptions we will build with

- One page, three tabs. All three panels stay mounted (only hidden), so each timer runs independently.
- Stopwatch shows hundredths of a second. Laps are listed newest first; best / worst colouring only with 2 or more laps.
- Countdown up to `99:59:59`. Inputs are editable only while idle. At zero: beep + message + Restart button.
- Pomodoro defaults 25 / 5 / 15, long break after every 4 finished focus sessions, overridable through a `config` prop. The next phase is **loaded but waits for Start**. **Skip** goes to the next phase and does **not** count a skipped focus session.
- No persistence across reload (named as an extension).
- Time comes from `Date.now()` (wall clock), so a timer is still right after the laptop wakes from sleep.

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Stopwatch: click Start	Runs from `00:00.00`; Start becomes Pause; Lap is enabled
Click Pause	Display freezes; buttons become Reset + Resume
Click Resume after 10 s paused	Continues from the frozen value; the paused 10 s are not counted
Click Lap while running	New row with that lap's time; the "current lap" row restarts from 0
Record 2+ laps	Fastest lap green, slowest red
Click Reset (paused)	Back to `00:00.00`, laps cleared, Lap disabled
Countdown: all fields 0	Start disabled, hint "Set a time greater than zero"
Type `75` in Minutes, leave the field	Value becomes `59`
Press Enter in an input	Starts the countdown (form submit)
Start a 5:00 countdown	Shows `05:00`, then `04:59` after one full second
Countdown reaches zero	Stops at `00:00` (red), beep, "Time's up!" announced, Restart button
Click Reset on the countdown	Back to the inputs, with the last values kept
Pomodoro: click Start	"Focus 25:00" counts down; tab title shows `24:59 · Focus`
A focus session ends	Beep, completed +1, one more dot filled, "Short break 05:00" loaded and idle
The 4th focus session ends	"Long break 15:00" loaded, all 4 dots filled
Click Skip during focus	Short break loaded; completed count unchanged
Switch tab while a timer runs	It keeps running; switching back shows the correct value
Hide the browser tab for 10 min	Value is right on return; a countdown ended on time and beeped

This table is the implementation checklist and the manual test plan.

---

3. Design the architecture

Module	Responsibility
`App`	Tab bar; keeps all three panels mounted and hides the inactive ones
`Stopwatch`	Stopwatch screen: display, controls, lap list
`Countdown`	Duration inputs, countdown display, done state
`Pomodoro`	Focus / break cycle on top of a countdown, tab title, session dots
`Button`	Shared pill button (primary / secondary)
`useClock(tickMs)`	The engine: start / pause / reset, elapsed ms from timestamps
`useCountdown(durationMs, onComplete)`	`useClock` + remaining time + exact end detection
`utils/clock.js`	Pure reducer for the clock state
`utils/format.js`	Time formatting and input clamping
`utils/pomodoro.js`	Default config and the pure "what comes next" rule
`utils/beep.js`	Short tone with Web Audio

```
App (active tab)
├── Stopwatch ── useClock(30)
├── Countdown ── useCountdown(duration, beep) ── useClock(200)
└── Pomodoro ─── useCountdown(phase duration, goNext) ── useClock(200)

useClock ── clockReducer + getElapsed (utils/clock.js)
all screens ── format.js, Button
```

Start simple: one `useClock` hook is enough for the stopwatch. `useCountdown` is split out only when the second screen needs "duration minus elapsed" and an end event. Pomodoro needs no new engine at all: it is a countdown whose duration changes with the phase.

---

4. Design the state / data model

State or field	Purpose
`clock.status`	`'idle' | 'running' | 'paused'`
`clock.startedAt`	`Date.now()` when the current running segment began; `null` when not running
`clock.accumulated`	ms counted in all earlier running segments (before the last pause)
`now` (in `useClock`)	Timestamp of the latest tick; exists only to trigger a re-render

State or field	Purpose
`splits` (Stopwatch)	Total elapsed at each Lap press, oldest first
`parts` (Countdown)	Raw input strings `{ h, m, s }`
`cycle` (Pomodoro)	`{ phase: 'work' | 'shortBreak' | 'longBreak', completed }`
`active` (App)	Which tab is visible

```
export const initialClock = { status: 'idle', startedAt: null, accumulated: 0 };

export function getElapsed(clock, now) {
  if (clock.status !== 'running') return clock.accumulated;
  return clock.accumulated + Math.max(0, now - clock.startedAt);
}
```

Senior note: derive, don't store.

- `elapsed` is **never stored**: it is `accumulated + (now - startedAt)`. Storing it and adding 1 per tick is exactly what causes drift.
- `remaining = max(0, duration - elapsed)`, `done = paused && remaining === 0`, lap durations, best / worst lap, the Pomodoro duration (`config[phase]`), and the tab title are all derived.
- **Ownership:** each screen owns its own timer (so they run independently). `useClock` owns the clock; screens only call `start / pause / reset`. `App` owns only the active tab.

---

5. Design the data structures and contracts

There is no backend, so no API or mock is needed. The contracts are the hook and function signatures.

Clock actions (pure reducer)

```
{ type: 'start', now: 1700000000000 }  // idle/paused -> running; ignored if already running
{ type: 'pause', now: 1700000010000 }  // running -> paused; folds the segment into accumulated
{ type: 'reset' }                       // anything -> initialClock
```

`now` is passed in, not read inside the reducer, so the reducer stays pure and can be tested with fake numbers.

Hooks

```
useClock(tickMs) => { clock, status, elapsed, read, start, pause, reset }
// read(): exact elapsed at this instant (for Lap clicks)

useCountdown(durationMs, onComplete, tickMs = 200)
  => { status, remaining, done, start, pause, reset, restart }
```

Pure helpers

```
formatStopwatch(62345)       // "01:02.34"
formatCountdown(299001)      // "05:00"  (rounds up)
partsToMs({ h: '1', m: '2', s: '3' })  // 3723000
advance({ phase: 'work', completed: 3 }, true, 4)
// => { phase: 'longBreak', completed: 4 }
```

Example data

```
const DEFAULT_POMODORO = { work: 25, shortBreak: 5, longBreak: 15, longEvery: 4 }; // minutes

const splits = [12340, 25010, 39990];    // total time at each Lap press
// laps (derived) = [12340, 12670, 14980] -> Lap 1 best (green), Lap 3 worst (red)
```

Stable ids: a lap's key is its lap number. Laps are only appended and are cleared all at once on reset, so number `n` always means the same lap. That is why an index-based number is safe here, while it would not be safe in a list where items can be removed from the middle.

---

6. Trace the data flow

Countdown of 5:00 with one pause:

```
User clicks Start at t = 1 000
  -> act('start'): setNow(1000) + dispatch({ type: 'start', now: 1000 })
  -> clock = { running, startedAt: 1000, accumulated: 0 }
  -> effect 1: setInterval(setNow(Date.now()), 200)   (re-render only)
  -> effect 2: check(): left = 300 000 -> setTimeout(check, 300 000)
  -> render: elapsed = now - 1000, remaining = 300 000 - elapsed
  -> formatCountdown(remaining) -> "05:00", "04:59", ...
User clicks Pause at t = 61 000
  -> clock = { paused, startedAt: null, accumulated: 60 000 }
  -> both effects clean up: interval and timeout cleared
  -> display frozen at "04:00"
User clicks Resume at t = 100 000  (39 s paused, not counted)
  -> clock = { running, startedAt: 100 000, accumulated: 60 000 }
  -> effect 2: left = 240 000 -> setTimeout(check, 240 000)
At t = 340 000 the timeout fires
  -> check(): left <= 0 -> pause() -> accumulated = 300 000
  -> onComplete() -> beep()
  -> render: remaining = 0, done = true -> "00:00" red, "Time's up!" announced
```

---

7. Important technical challenges

Challenge 1: Timer drift

Problem. The obvious code counts ticks:

```
// BAD: assumes every tick is exactly 1000 ms apart
setInterval(() => setSeconds((s) => s + 1), 1000);
```

`setInterval` is a minimum delay, not a promise. A busy main thread delays ticks by a few ms each time; a hidden tab is throttled to one tick per second, and in Chrome after 5 minutes hidden, to about **one per minute**. Start a 25 minute Pomodoro, switch tabs, come back after 25 minutes: the counter may show only a few minutes passed.

Solution. Store *when* the timer started and compute elapsed from the clock. The tick only asks React to re-render; if a tick is late or skipped, the next one still shows the right value.

```
const [now, setNow] = useState(() => Date.now());
useEffect(() => {
  if (clock.status !== 'running') return;
  const id = setInterval(() => setNow(Date.now()), tickMs);
  return () => clearInterval(id);
}, [clock.status, tickMs]);

const elapsed = getElapsed(clock, now); // accumulated + (now - startedAt)
```

Challenge 2: Pause / resume without losing or gaining time

Problem. With only `startedAt`, pausing has nowhere to keep the time already counted, and resuming would either restart at zero or include the paused gap. Scenario: run 10 s, pause 5 s, resume, wait 3 s: the display must say **13 s**, not 18 s or 3 s. A second trap: on resume `startedAt` becomes "now", but the `now` state is still the timestamp of the last tick before the pause, which is *older*, so `now - startedAt` is negative for one render and the display jumps backwards.

Solution. Keep `accumulated` + `startedAt`. Pause folds the running segment into `accumulated`. Every action sets `now` and `startedAt` from the same timestamp, and `getElapsed` clamps the segment at 0.

```
case 'pause':
  if (state.status !== 'running') return state;
  return { status: 'paused', startedAt: null,
           accumulated: state.accumulated + (action.now - state.startedAt) };

const act = useCallback((type) => {
  const t = Date.now();
  setNow(t);                 // now >= startedAt, so no backwards jump on resume
  dispatch({ type, now: t });
}, []);
```

Guarding `start` while running and `pause` while paused also makes double clicks harmless.

Challenge 3: Ending on time, and showing the right number

Problem. Checking `remaining <= 0` inside the tick ties the end event to the tick rate. In a hidden tab the interval runs once a minute, so the Pomodoro ends (and beeps) up to a minute late. Separately, the display rounding is easy to get wrong: with `Math.floor`, a 5:00 countdown shows `04:59` immediately after Start and shows `00:00` for the whole last second while time is still left.

Solution. Schedule **one** `setTimeout` for the exact time left. A single timeout is not part of a chain of timers, so it avoids the heavy throttling that hits `setInterval`. When it fires, re-check and reschedule if it came early. For display: the countdown rounds **up**, the stopwatch rounds **down** (a stopwatch must never show time that has not passed yet).

```
useEffect(() => {
  if (clock.status !== 'running') return;
  let id;
  const check = () => {
    const left = durationMs - getElapsed(clock, Date.now());
    if (left > 0) { id = setTimeout(check, left); return; }
    pause();
    onCompleteRef.current?.();
  };
  check();
  return () => clearTimeout(id);
}, [clock, durationMs, pause]);

const total = Math.max(0, Math.ceil(ms / 1000));  // countdown: 299 001 ms -> "05:00"
```

`done` is derived as `paused && remaining === 0`, so there is no extra "finished" flag to keep in sync.

Challenge 4: Stale values (lap times and the completion callback)

Problem. (a) The rendered `elapsed` can be up to one tick old (30 ms). A Lap click that saves `elapsed` records the time of the last render, not of the click. (b) `onComplete` is a new function every render (Pomodoro's one reads the current phase). If it is a dependency of the timeout effect, the timeout is cleared and rescheduled on every tick; if it is left out, the timeout calls an old closure.

Solution. (a) `read()` computes elapsed from `Date.now()` at the moment of the click. (b) Keep the latest callback in a ref; the effect calls `onCompleteRef.current`. In Pomodoro, the state update uses the functional form so it never reads a stale `cycle`.

```
const read = useCallback(() => getElapsed(clock, Date.now()), [clock]);
<Button onClick={() => setSplits((s) => [...s, read()])}>Lap</Button>

const onCompleteRef = useRef(onComplete);
useEffect(() => { onCompleteRef.current = onComplete; });
```

```
setCycle((c) => advance(c, finished, config.longEvery)); // Pomodoro
```

8. Implement in the correct order

#	Step	Working result
1	Vite + Tailwind, `App` with three tabs and placeholder panels (kept mounted, `hidden`)	Tabs switch
2	`clockReducer` + `getElapsed` (pure)	Can be checked with numbers in the console
3	`useClock`: reducer, `now` tick, `act()`	Engine ready
4	Stopwatch display + Start / Pause / Resume / Reset	A working stopwatch
5	Laps: `splits`, derived laps, current lap row, best / worst colours	Full stopwatch
6	`formatCountdown` (ceil) + `useCountdown` with the end timeout	Countdown engine
7	Countdown inputs as a form, `clampPart` on blur, zero-duration guard	Can set and start a countdown
8	Countdown running / paused / done views, Restart	Full countdown
9	`beep()` + `aria-live` "Time's up!"	End is heard and announced
10	Pomodoro: `cycle` state, `advance()`, auto-load next phase, Skip	Full Pomodoro cycle
11	Tab title, session dots, completed count	Pomodoro polish
12	Edge-case pass (table below), hidden-tab test	Done

After step 4 there is already something to demo. Each step after that adds one layer that works on its own.

9. Cover all states and edge cases

State / edge case	Handling
Idle (initial)	Stopwatch `00:00.00`, Lap disabled; Countdown shows inputs; Pomodoro `25:00`, Reset disabled
Running	Ticking display, Pause button, timers active
Paused	Display frozen, no interval or timeout running; Resume + Reset
Done (countdown)	`00:00` in red, "Time's up!" (`aria-live`), beep, Restart
Phase change (Pomodoro)	Next phase loaded idle; phase name announced (`aria-live="polite"`)
Empty (no laps)	Only the "current lap" row; colours only with 2+ laps; equal laps are not coloured
Invalid input	`""`, `abc`, `-3` become 0; `2.7` becomes 2; `75` becomes 59; hours capped at 99
Zero duration	Start disabled + hint; Enter does nothing

State / edge case	Handling
Double click Start / Pause	Reducer ignores start-while-running and pause-while-paused
Resume after a long pause	Paused time not counted (`accumulated` + new `startedAt`)
Lap click between ticks	`read()` uses `Date.now()`, so the lap is exact
Hidden browser tab	Value is derived, so it is right on return; end timeout fires on time
Laptop sleep	`Date.now()` keeps counting real time; timeout fires on wake
Switch mode tab	Panels stay mounted, so timers continue
Stopwatch over 1 hour	Shows `1:00:00.00`
Skip during a running focus	Timer reset, short break loaded, focus not counted
Audio blocked	`beep()` swallows the error; the visual done state still shows
Unmount while running	Effects clear the interval and timeout; no leaks
React StrictMode double effects	Every effect has a cleanup, so the second run is safe

10. Final structure

```
_69_stopwatch_countdown_pomodoro/
├── index.html
├── package.json
├── vite.config.js
└── src/
    ├── main.jsx
    ├── index.css
    ├── App.jsx                tabs, keeps panels mounted
    ├── components/
    │   ├── Button.jsx
    │   ├── Stopwatch.jsx
    │   ├── Countdown.jsx
    │   └── Pomodoro.jsx
    ├── hooks/
    │   ├── useClock.js        start / pause / reset, elapsed from timestamps
    │   └── useCountdown.js    remaining, done, end timeout
    └── utils/
        ├── clock.js           pure reducer + getElapsed
        ├── format.js          formatStopwatch, formatCountdown, clampPart, partsToMs
        ├── pomodoro.js        DEFAULT_POMODORO, PHASE_LABEL, advance
        └── beep.js            Web Audio tone
```

11. Full working code

vite.config.js

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react-swc';
import tailwindcss from '@tailwindcss/vite';

export default defineConfig({
  plugins: [react(), tailwindcss()],
});
```

src/index.css

```
@import "tailwindcss";
```

src/main.jsx

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import './index.css';
import App from './App';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

src/utils/clock.js

```
// The clock never counts ticks: elapsed time is always derived from timestamps.
export const initialClock = { status: 'idle', startedAt: null, accumulated: 0 };

export function clockReducer(state, action) {
  switch (action.type) {
    case 'start':
      if (state.status === 'running') return state;
      return { ...state, status: 'running', startedAt: action.now };
    case 'pause':
      if (state.status !== 'running') return state;
      return {
        status: 'paused',
        startedAt: null,
        accumulated: state.accumulated + (action.now - state.startedAt),
```

```
      };
    case 'reset':
      return initialClock;
    default:
      return state;
  }
}

export function getElapsed(clock, now) {
  if (clock.status !== 'running') return clock.accumulated;
  return clock.accumulated + Math.max(0, now - clock.startedAt);
}
```

src/utils/format.js

```
const pad = (n) => String(n).padStart(2, '0');

// Stopwatch rounds down: it must never show time that has not passed yet. "01:02.34", "1:00:00.00"
export function formatStopwatch(ms) {
  const cs = Math.max(0, Math.floor(ms / 10));
  const h = Math.floor(cs / 360000);
  const m = Math.floor(cs / 6000) % 60;
  const s = Math.floor(cs / 100) % 60;
  return `${h ? `${h}:` : ''}${pad(m)}:${pad(s)}.${pad(cs % 100)}`;
}

// Countdown rounds up: 05:00 shows at the start and 00:00 only when time is really up.
export function formatCountdown(ms) {
  const total = Math.max(0, Math.ceil(ms / 1000));
  const h = Math.floor(total / 3600);
  const m = Math.floor(total / 60) % 60;
  return `${h ? `${pad(h)}:` : ''}${pad(m)}:${pad(total % 60)}`;
}

// Input boxes hold raw strings; turn "", "-3", "2.7", "75" into a safe whole number.
export const clampPart = (value, max) => Math.min(max, Math.max(0, Math.floor(Number(value)) || 0));

export const partsToMs = ({ h, m, s }) =>
  (clampPart(h, 99) * 3600 + clampPart(m, 59) * 60 + clampPart(s, 59)) * 1000;
```

src/utils/pomodoro.js

```
// Durations in minutes; a long break comes after every `longEvery` finished focus sessions.
export const DEFAULT_POMODORO = { work: 25, shortBreak: 5, longBreak: 15, longEvery: 4 };

export const PHASE_LABEL = { work: 'Focus', shortBreak: 'Short break', longBreak: 'Long break' };
```

```
// finished = the timer reached zero; false = the user pressed Skip (a skipped focus is not counted).
export function advance({ phase, completed }, finished, longEvery) {
  if (phase !== 'work') return { phase: 'work', completed };
  if (!finished) return { phase: 'shortBreak', completed };
  const count = completed + 1;
  return { phase: count % longEvery === 0 ? 'longBreak' : 'shortBreak', completed: count };
}
```

src/utils/beep.js

```
let ctx;

// A short tone from the Web Audio API, so there is no sound file to load.
export function beep(ms = 400, freq = 880) {
  try {
    ctx ??= new AudioContext();
    ctx.resume().catch(() => {});
    const osc = ctx.createOscillator();
    osc.frequency.value = freq;
    osc.connect(ctx.destination);
    osc.start();
    osc.stop(ctx.currentTime + ms / 1000);
  } catch {
    // Audio blocked or unsupported: the on-screen "done" state still shows.
  }
}
```

src/hooks/useClock.js

```
import { useCallback, useEffect, useReducer, useState } from 'react';
import { clockReducer, getElapsed, initialClock } from '../utils/clock';

// Stopwatch engine: start / pause / reset, elapsed ms derived from Date.now().
export function useClock(tickMs = 1000) {
  const [clock, dispatch] = useReducer(clockReducer, initialClock);
  const [now, setNow] = useState(() => Date.now());

  // The interval only triggers a re-render; a late or skipped tick cannot add drift.
  useEffect(() => {
    if (clock.status !== 'running') return;
    const id = setInterval(() => setNow(Date.now()), tickMs);
    return () => clearInterval(id);
  }, [clock.status, tickMs]);

  const act = useCallback((type) => {
```

```
    const t = Date.now();
    setNow(t); // keeps `now` >= startedAt, so the display never jumps on resume
    dispatch({ type, now: t });
  }, []);

  const start = useCallback(() => act('start'), [act]);
  const pause = useCallback(() => act('pause'), [act]);
  const reset = useCallback(() => act('reset'), [act]);
  // Exact value at the moment of a click (the rendered value can be one tick old).
  const read = useCallback(() => getElapsed(clock, Date.now()), [clock]);

  return { clock, status: clock.status, elapsed: getElapsed(clock, now), read, start, pause, reset };
}
```

src/hooks/useCountdown.js

```
import { useCallback, useEffect, useRef } from 'react';
import { useClock } from './useClock';
import { getElapsed } from '../utils/clock';

// Countdown = a clock compared with a duration. The end is detected by one timeout, not by ticks.
export function useCountdown(durationMs, onComplete, tickMs = 200) {
  const { clock, status, elapsed, start, pause, reset } = useClock(tickMs);
  const remaining = Math.max(0, durationMs - elapsed);
  const done = status === 'paused' && remaining === 0;

  // Latest callback without restarting the timeout every render.
  const onCompleteRef = useRef(onComplete);
  useEffect(() => {
    onCompleteRef.current = onComplete;
  });

  useEffect(() => {
    if (clock.status !== 'running') return;
    let id;
    const check = () => {
      const left = durationMs - getElapsed(clock, Date.now());
      if (left > 0) {
        id = setTimeout(check, left); // one timer for the end time; re-check in case it fires early
        return;
      }
      pause();
      onCompleteRef.current?.();
    };
    check();
    return () => clearTimeout(id);
  }, [clock, durationMs, pause]);
```

```
  const restart = useCallback(() => {
    reset();
    start();
  }, [reset, start]);

  return { status, remaining, done, start, pause, reset, restart };
}
```

src/components/Button.jsx

```
const VARIANTS = {
  primary: 'bg-indigo-600 text-white hover:bg-indigo-500',
  secondary: 'bg-slate-200 text-slate-800 hover:bg-slate-300',
};

export default function Button({ variant = 'primary', ...props }) {
  return (
    <button
      type="button"
      className={`min-w-24 rounded-full px-5 py-2 font-medium disabled:cursor-not-allowed
disabled:opacity-40 ${VARIANTS[variant]}`}
      {...props}
    />
  );
}
```

src/components/Stopwatch.jsx

```
import { useState } from 'react';
import { useClock } from '../hooks/useClock';
import { formatStopwatch } from '../utils/format';
import Button from './Button';

export default function Stopwatch() {
  const { status, elapsed, read, start, pause, reset } = useClock(30);
  const [splits, setSplits] = useState([]); // total elapsed at each Lap press, oldest first

  const laps = splits.map((split, i) => split - (splits[i - 1] ?? 0));
  const best = Math.min(...laps);
  const worst = Math.max(...laps);
  const currentLap = elapsed - (splits.at(-1) ?? 0);

  function lapColor(lap) {
    if (laps.length < 2 || best === worst) return '';
    if (lap === best) return 'text-emerald-600';
```

```
  return lap === worst ? 'text-rose-600' : '';
}

function handleReset() {
  reset();
  setSplits([]);
}

return (
  <div className="text-center">
    <p role="timer" className="font-mono text-6xl tabular-nums">
      {formatStopwatch(elapsed)}
    </p>

    <div className="mt-6 flex justify-center gap-4">
      {status === 'paused' ? (
        <Button variant="secondary" onClick={handleReset}>Reset</Button>
      ) : (
        <Button
          variant="secondary"
          onClick={() => setSplits((s) => [...s, read()])}
          disabled={status !== 'running'}
        >
          Lap
        </Button>
      )}
      {status === 'running' ? (
        <Button onClick={pause}>Pause</Button>
      ) : (
        <Button onClick={start}>{status === 'paused' ? 'Resume' : 'Start'}</Button>
      )}
    </div>

    {status !== 'idle' && (
      <ol className="mt-6 divide-y divide-slate-200 font-mono text-sm tabular-nums">
        <li className="flex justify-between py-2 text-slate-500">
          <span>Lap {laps.length + 1}</span>
          <span>{formatStopwatch(currentLap)}</span>
        </li>
        {laps
          .map((lap, i) => ({ n: i + 1, lap }))
          .reverse()
          .map(({ n, lap }) => (
            <li key={n} className={`flex justify-between py-2 ${lapColor(lap)}`}>
              <span>Lap {n}</span>
              <span>{formatStopwatch(lap)}</span>
            </li>
```

```
            ))}
        </ol>
      )}
    </div>
  );
}
```

`src/components/Countdown.jsx`

```
import { useState } from 'react';
import { useCountdown } from '../hooks/useCountdown';
import { clampPart, formatCountdown, partsToMs } from '../utils/format';
import { beep } from '../utils/beep';
import Button from './Button';

const FIELDS = [
  { key: 'h', label: 'Hours', max: 99 },
  { key: 'm', label: 'Minutes', max: 59 },
  { key: 's', label: 'Seconds', max: 59 },
];

export default function Countdown() {
  const [parts, setParts] = useState({ h: '0', m: '5', s: '0' });
  const durationMs = partsToMs(parts);
  const { status, remaining, done, start, pause, reset, restart } = useCountdown(durationMs, beep);

  function handleSubmit(e) {
    e.preventDefault();
    if (durationMs > 0) start();
  }

  return (
    <div className="text-center">
      {status === 'idle' ? (
        <form onSubmit={handleSubmit}>
          <div className="flex justify-center gap-3">
            {FIELDS.map(({ key, label, max }) => (
              <label key={key} className="flex flex-col text-xs text-slate-500">
                {label}
                <input
                  type="number"
                  inputMode="numeric"
                  min="0"
                  max={max}
                  value={parts[key]}
                  onChange={(e) => setParts((p) => ({ ...p, [key]: e.target.value }))}
                  onBlur={() => setParts((p) => ({ ...p, [key]: String(clampPart(p[key], max)) }))}
```

```
                  className="mt-1 w-20 rounded-lg border border-slate-300 p-2 text-center font-mono
text-2xl text-slate-800"
                />
              </label>
            ))}
          </div>
          <p className="mt-2 h-5 text-sm text-rose-600">
            {durationMs === 0 ? 'Set a time greater than zero' : ''}
          </p>
          <div className="mt-4">
            <Button type="submit" disabled={durationMs === 0}>Start</Button>
          </div>
        </form>
      ) : (
        <>
          <p role="timer" className={`font-mono text-6xl tabular-nums ${done ? 'text-rose-600' : ''}`}>
            {formatCountdown(remaining)}
          </p>
          <div className="mt-6 flex justify-center gap-4">
            <Button variant="secondary" onClick={reset}>Reset</Button>
            {done ? (
              <Button onClick={restart}>Restart</Button>
            ) : status === 'running' ? (
              <Button onClick={pause}>Pause</Button>
            ) : (
              <Button onClick={start}>Resume</Button>
            )}
          </div>
        </>
      )}
      <p aria-live="assertive" className="mt-4 h-6 font-medium text-rose-600">
        {done ? "Time's up!" : ''}
      </p>
    </div>
  );
}
```

src/components/Pomodoro.jsx

```
import { useEffect, useState } from 'react';
import { useCountdown } from '../hooks/useCountdown';
import { formatCountdown } from '../utils/format';
import { DEFAULT_POMODORO, PHASE_LABEL, advance } from '../utils/pomodoro';
import { beep } from '../utils/beep';
import Button from './Button';

export default function Pomodoro({ config = DEFAULT_POMODORO }) {
```

```
  const [cycle, setCycle] = useState({ phase: 'work', completed: 0 });
  const timer = useCountdown(config[cycle.phase] * 60_000, () => {
    beep();
    goNext(true);
  });

  // Load the next phase and wait for Start (no surprise auto-start).
  function goNext(finished) {
    setCycle((c) => advance(c, finished, config.longEvery));
    timer.reset();
  }

  const label = formatCountdown(timer.remaining);
  const isWork = cycle.phase === 'work';
  const filled = cycle.phase === 'longBreak' ? config.longEvery : cycle.completed % config.longEvery;

  // Remaining time in the browser tab title, visible while the user is in another tab.
  useEffect(() => {
    if (timer.status === 'idle') return;
    const prev = document.title;
    document.title = `${label} · ${PHASE_LABEL[cycle.phase]}`;
    return () => {
      document.title = prev;
    };
  }, [label, cycle.phase, timer.status]);

  return (
    <div className="text-center">
      <p aria-live="polite" className={`mb-2 font-semibold ${isWork ? 'text-indigo-600' : 'text-
emerald-600'}`}>
        {PHASE_LABEL[cycle.phase]}
      </p>
      <p role="timer" className="font-mono text-6xl tabular-nums">{label}</p>

      <div className="mt-6 flex justify-center gap-4">
        <Button variant="secondary" onClick={timer.reset} disabled={timer.status ===
'idle'}>Reset</Button>
        {timer.status === 'running' ? (
          <Button onClick={timer.pause}>Pause</Button>
        ) : (
          <Button onClick={timer.start}>{timer.status === 'paused' ? 'Resume' : 'Start'}</Button>
        )}
        <Button variant="secondary" onClick={() => goNext(false)}>Skip</Button>
      </div>

      <div aria-hidden="true" className="mt-6 flex justify-center gap-2">
        {Array.from({ length: config.longEvery }, (_, i) => (
```

```
          <span key={i} className={`size-3 rounded-full ${i < filled ? 'bg-indigo-600' : 'bg-slate-
200'}`} />
        ))}
      </div>
      <p className="mt-2 text-sm text-slate-500">Focus sessions completed: {cycle.completed}</p>
    </div>
  );
}
```

src/App.jsx

```
import { useState } from 'react';
import Stopwatch from './components/Stopwatch';
import Countdown from './components/Countdown';
import Pomodoro from './components/Pomodoro';

const MODES = [
  { id: 'stopwatch', label: 'Stopwatch', Panel: Stopwatch },
  { id: 'countdown', label: 'Countdown', Panel: Countdown },
  { id: 'pomodoro', label: 'Pomodoro', Panel: Pomodoro },
];

export default function App() {
  const [active, setActive] = useState('stopwatch');

  return (
    <main className="mx-auto max-w-md p-6 text-slate-800">
      <h1 className="mb-4 text-center text-2xl font-bold">Timers</h1>

      <div role="tablist" aria-label="Timer mode" className="mb-8 flex rounded-full bg-slate-100 p-1">
        {MODES.map(({ id, label }) => (
          <button
            key={id}
            id={`tab-${id}`}
            type="button"
            role="tab"
            aria-selected={active === id}
            aria-controls={`panel-${id}`}
            onClick={() => setActive(id)}
            className={`flex-1 rounded-full py-2 text-sm font-medium ${active === id ? 'bg-white
shadow' : 'text-slate-500'}`}
          >
            {label}
          </button>
        ))}
      </div>
```

```
      {/* Panels stay mounted (only hidden) so a running timer keeps going when you switch tabs. */}
      {MODES.map(({ id, Panel }) => (
        <section key={id} id={`panel-${id}`} role="tabpanel" aria-labelledby={`tab-${id}`} hidden=
{active !== id}>
          <Panel />
        </section>
      ))}
    </main>
  );
}
```

---

12. Wrap-up

Edge cases handled

- No drift: time is always derived from `Date.now()`; ticks only re-render.
- Pause / resume is exact, including double clicks and long pauses.
- The countdown ends on time even in a hidden tab (single end timeout) and displays with the right rounding.
- Lap times are taken at the click, not from the last render.
- Bad input is clamped; zero duration cannot start; blocked audio is harmless.
- Timers keep running across mode tabs; everything is cleaned up on unmount.

Trade-offs to mention

- **`Date.now()` vs `performance.now()`:** `Date.now()` keeps counting through laptop sleep and could later be saved for persistence, but it jumps if the user changes the system clock. `performance.now()` is monotonic but resets on reload and may pause during sleep.
- **`setInterval` (30 / 200 ms) vs `requestAnimationFrame`:** rAF gives the smoothest stopwatch but stops in hidden tabs and runs at 60-120 fps. A fixed interval is simpler and cheaper; the stopwatch at 30 ms looks smooth enough.
- **All panels mounted:** keeps timers alive across tabs at the cost of a hidden running panel still re-rendering. Lifting timer state up (or into a store) would avoid that but adds code.
- **No auto-start of the next Pomodoro phase:** avoids starting a break when the user stepped away; easy to add as an option.

What to add with more time

- **Tests:** unit tests for `clockReducer`, `formatCountdown`, `advance` (pure, no React); hook tests with `vi.useFakeTimers()` + `vi.setSystemTime()` to fast-forward 25 minutes.
- **Persistence:** save `{ status, startedAt, accumulated }` in `localStorage`; on load the timer is correct immediately because elapsed is derived from timestamps.
- **Notifications:** the Notification API for "Focus done" when the tab is hidden (asked for permission on the first Start).
- **Settings UI** for Pomodoro durations, auto-start toggle, keyboard shortcuts (Space = start / pause, L = lap, R = reset).

- **Many timers on one page:** one shared ticker (a small store + `useSyncExternalStore`) instead of one interval per component.

---

13. Counter questions the interviewer may ask

Requirements and approach

Q1. Why not just `setInterval(() => setSeconds(s => s + 1), 1000)`?

Because it assumes every tick is exactly 1000 ms apart. Ticks are delayed by a busy main thread and heavily throttled in hidden tabs, so the error adds up. Deriving elapsed from timestamps makes each render correct on its own, no matter how many ticks were late or skipped.

Q2. Why `Date.now()` and not `performance.now()`?

`Date.now()` is wall-clock time: it keeps going during laptop sleep and its values can be saved and compared after a reload. `performance.now()` is monotonic (never jumps if the system clock changes) but starts from 0 on every page load. For a Pomodoro, "25 real minutes" matters more, so `Date.now()`; for measuring code speed, `performance.now()`.

Q3. What if the user changes the system clock while a timer runs?

With `Date.now()` the elapsed time jumps by the same amount; `getElapsed` clamps the running segment at 0 so it can never go negative. If that matters, use `performance.now()` for the running segment and keep `Date.now()` only for persistence.

State design

Q4. Why not store `elapsed` in state?

It would be a second source of truth that has to be updated by ticks, which brings drift back. `elapsed` is a pure function of `accumulated`, `startedAt` and the current time, so it is computed on every render.

Q5. Why `useReducer` for the clock instead of three `useState` calls?

`status`, `startedAt` and `accumulated` always change together (pause changes all three). A reducer updates them atomically, keeps the rules in one pure function and is trivial to unit test with fake timestamps.

Q6. Why store lap *splits* (totals) instead of lap durations?

A split is exactly what the clock reads at the click, with no subtraction that could pile up rounding. Durations, best and worst are derived. It also makes the "current lap" row simple: `elapsed - lastSplit`.

Hard parts

Q7. What happens if the tab is hidden for 30 minutes?

The interval barely runs, but nothing depends on it. When the user comes back, the next render computes the right elapsed value. The countdown's end was detected by its own single timeout, which fires on time (or within about a second), and the beep plays then.

Q8. Why one `setTimeout` for the end instead of checking in each tick?

Chrome throttles chains of timers (like `setInterval`) in hidden tabs to about once a minute, so a tick-based check can end a Pomodoro up to a minute late. A single timeout set for the exact remaining time is not in such a chain. The callback re-checks and reschedules in case it fires early.

Q9. Why `Math.ceil` for the countdown but `Math.floor` for the stopwatch?

A countdown should show `05:00` right after Start and `00:00` only when time is really up; flooring would show `04:59` at once and sit on `00:00` for the last second. A stopwatch must not show a second that has not fully passed, so it floors.

Q10. Why 30 ms and 200 ms ticks? Why not `requestAnimationFrame`?

The stopwatch shows hundredths, so ~30 fps looks smooth. The countdown shows whole seconds; a 200 ms tick keeps the visible change within 0.2 s of the real second boundary with 5 renders per second. rAF would render 60-120 times per second for a number that changes once a second, and it stops completely in hidden tabs.

Q11. Why keep `onComplete` in a ref?

The callback is a new function on every render. As an effect dependency it would clear and reschedule the end timeout on every tick; left out, the timeout would call an old closure. The ref always holds the latest one without touching the timeout.

Implementation

Q12. Why keep all three panels mounted? What does it cost?

Unmounting a panel would destroy its state, so switching to Countdown would kill a running stopwatch. Hiding keeps them alive. The cost is that a hidden running panel still re-renders on its tick; if that mattered, I would lift each timer's state up or into a small store and render only the visible one.

Q13. Does React StrictMode's double effect run break anything?

No. Every effect returns a cleanup (`clearInterval`, `clearTimeout`, restore `document.title`), so mount, unmount, mount ends in the same state as a single mount. `act()` is an event handler, not an effect, so it never runs twice.

Q14. How would you make the timers survive a page reload?

Save `{ status, startedAt, accumulated }` (and Pomodoro's `cycle`) to `localStorage` on every action, and use it as the reducer's initial state. Because elapsed is derived from timestamps, a running timer shows the right value right after reload. A countdown that ended while the page was closed would show done at once.

Q15. How do you tell the user the Pomodoro ended if they are in another tab?

The tab title already shows the time. For a real alert, use the Notification API (ask permission on a user click such as Start) and show a notification in `onComplete` when `document.hidden` is true. The beep also works, because the user already interacted with the page.

Q16. Why `role="timer"` and not `aria-live` on the display?

`aria-live` on a value that changes 5-30 times a second would flood a screen reader. `role="timer"` marks it as a timer that is not announced automatically. Only meaningful events ("Time's up!", a new phase) go into `aria-live` regions.

Performance and scale

Q17. What if there are 100 timers on one page (e.g. auction countdowns)?

100 intervals and 100 independent re-renders is wasteful. Use one shared ticker: a tiny store that updates `now` once per second and lets components subscribe through `useSyncExternalStore`. Each card derives its remaining time from its own end timestamp. For long lists, also virtualize so off-screen cards don't render.

Q18. Would a Web Worker make it more accurate?

Worker timers are throttled less in hidden tabs, so it can help when you must *run code* exactly on time in the background. It does not help the displayed value, which is already exact because it is derived. For this app the single end timeout is enough; a worker adds messaging complexity.

Trade-offs and "why not X?"

Q19. Why not use a library like `react-timer-hook`?

In an interview the point is to show the drift and pause / resume reasoning. In production a well-tested library is fine, but `useClock` is about 25 lines and has no dependency, so I would keep it.

Q20. Why does the next Pomodoro phase not start automatically?

If the user left the desk, an auto-started break would end before they return and the next focus would start without them. Loading the phase and waiting for Start is the safer default; an "auto-start" setting is a one-line change in `goNext` (call `timer.start()` after `reset()`).

Q21. How would you test this?

The pure parts (`clockReducer`, `getElapsed`, format functions, `advance`) are tested with plain numbers, no React. For the hooks and screens, use React Testing Library with `vi.useFakeTimers()` and `vi.setSystemTime()`: click Start, advance 5 minutes, and assert "Time's up!" is shown and the beep mock was called once.